SG DMA — Sanity Test

항목	내용
문서 ID	SGDMA-SANITY-rel-v2.0.0
날짜	2026-10-16 (v2.0.0 개정) · 2026-10-06 (v1.0.0 인계)
작성자	hrcho (v1.0.0)
대상 DUT	`rtl/sg_dma/sgdma_top.v`
구현	`tb/sg_dma/tb_sg_dma.sv`
구성	**TC 7건 / 체크 46건**
최종 실행	**2026-10-06 — 46/46 PASS** (시뮬레이션 시간 6,305 ns)
상태	**검증팀 인계본** — v2.0.0은 **문서만** 고쳤습니다. TB 코드·체크·기대값·로그는 v1.0.0과 같습니다
원본문서	$repo/docs/ref_hrcho/261006-claude-sgdma_sanity_test-001-hrcho.md

관련 문서

문서	역할
`SGDMA-SOR-rel-v2.0.0`	**검증 요구사항** — 테스트 15개(T-01 ~ T-15). 이 문서의 SOR 연결은 모두 이 판 기준
`SGDMA-SFR-rel-v2.0.0`	**레지스터 설명** — 이 문서의 모든 기대값 근거
`SGDMA-SOR-INTEGRATION-v1.0`	SOR 축소안 통합 문서. 7장이 이번 개정의 근거
`$repo/docs/ref_hrcho/260930-claude-rtl_design_crosscheck-hrcho.md`	RTL 구성 교차 검증

> 이 문서에 적힌 SFR 절 번호(예: `[4.1.1]`, `4.6.2`)는 SFR **rel-v1.0.0**의 번호입니다. SFR v2.0.0에서 절 번호가 바뀌면 부록 B를 같이 고칩니다.

이 문서를 처음 읽는 분께

가장 먼저 5장 커버 범위와 한계를 읽으십시오.

> ⚠ **46/46 PASS는 "이 46개가 통과했다"는 뜻입니다. "레지스터와 DMA가 전부 정상이다"는 뜻이 아닙니다.** 예를 들어 `SAN-003`은 리셋값을 확인하지만, SFR 부록 A에 있는 약 30개 중 **10개만** 봅니다. 나머지는 일부러 검증팀(SOR)에 남긴 것입니다. 5장에 전부 적었습니다.

이 구분을 놓치면 "이미 확인된 항목"으로 읽고 건너뛰게 됩니다. 그것이 이 문서가 막으려는 가장 큰 위험입니다.

목차

- 1. Sanity 처음 보는 분께
- 2. 구성
- 3. TC 상세

- 4. 실행 방법
- 5. 커버 범위와 한계
- 6. TB 구조
- 7. 유지보수
- 부록 A — 실측 로그 전문
- 부록 B — 체크 ↔ SFR 근거 대조표
- 부록 C — 설계 결정 반영 이력
- 개정 이력

---

1. Sanity 처음 보는 분께

1.1 비유로 이해하기

Sanity Test는 **RTL을 고칠 때마다 돌리는 빠른 건강검진**입니다. 체온·혈압·맥박처럼 "살아 있나, 크게 고장 났나"만 몇 초 만에 봅니다. 정밀검사(CT, 혈액 정밀분석)는 따로 있습니다. SG DMA에서는 그것이 **SOR의 테스트 15개**(T-01 ~ T-15)입니다. 건강검진이 정상이어도 정밀검사를 건너뛰지 않듯, **Sanity가 통과해도 SOR 테스트는 따로 돌립니다.** 반대로 건강검진에서 이상이 나오면, 정밀검사로 넘어가기 전에 그것부터 고칩니다.

SG DMA 자체가 처음이라면 `SGDMA-SOR-rel-v2.0.0` 1장("SG DMA 처음 보는 분께")을 먼저 읽으십시오. descriptor, 체인, gather/scatter, 표준 순서를 거기서 설명합니다.

1.2 Sanity의 역할: 회귀 게이트

Sanity의 목적은 **"RTL이 완전히 깨졌는가"를 수 초 안에 판별**하는 것입니다. 기능을 폭넓게 검증하는 것이 아닙니다. **다음 단계로 넘어가도 되는지를 판정하는 관문**(회귀 게이트)입니다.

RTL을 고칠 때마다 돌리므로, 아래 네 조건을 지킵니다.

조건	이 구현	이유
빠를 것	시뮬레이션 시간 **6,305 ns**, 실행 수 초	느리면 아무도 안 돌립니다
자동 판정	체크마다 `[ ok ]` / `[FAIL]`로 판정하고 요약에서 집계	사람이 로그를 읽고 판단하면 게이트가 아닙니다
비정상 종료 코드	실패하면 `$fatal(1)`	CI(자동 빌드·검사 시스템)나 스크립트가 결과를 알아야 합니다
watchdog (시간 초과 감시)	전역 `#2_000_000` 타임아웃	핸드셰이크가 멈추면 회귀가 **영원히 끝나지 않습니다**

> ⚠️ **watchdog이 특히 중요합니다.** 기능 TB는 멈추면 사람이 보고 끄면 됩니다. 회귀는 사람 없이 돕니다. 타임아웃 없이 멈추면(hang) 그날 회귀 전체가 멎습니다.

```
        // global watchdog - a hung handshake must not hang the regression
        initial begin
          #2_000_000;
          $display("\n[FAIL] GLOBAL TIMEOUT - simulation did not finish");
          $fatal(1, "global watchdog");
        end
```

전송 하나를 기다릴 때도 따로 타임아웃이 있습니다(`wait_irq`, 기본 50,000 사이클). 이쪽은 **그 TC의 FAIL 체크로 기록**됩니다. 그래서 어느 TC에서 멈췄는지 로그에 남습니다.

1.3 Sanity가 하지 않는 것

아래는 Sanity의 역할이 아닙니다. 이 문서도 그렇다고 주장하지 않습니다.

하지 않는 것	설명	대신 하는 곳
커버리지 측정	코드/기능 커버리지를 재지 않습니다	SOR에서도 제외 (`COV-001`)
전수 검사	레지스터 전체를 읽지 않습니다 (5.2)	—
기능 검증	경계 조건·에러 주입·랜덤 조합은 범위 밖입니다	SOR T-05 ~ T-15
성능 측정	대역폭·지연(latency)을 보지 않습니다	—

이 구분은 팀 합의 사항입니다(2026-10-06). 리셋값 미확인 항목을 Sanity에 추가하는 방안이 논의되었습니다. 하지만 **Sanity는 회귀 게이트 역할에 한정**하기로 했습니다. 리셋 관련은 SOR에서도 다루지 않기로 정리되었습니다 (부록 C).

1.4 SOR과의 관계

	Sanity (이 문서)	SOR (`SGDMA-SOR-rel-v2.0.0`)
목적	RTL이 크게 깨졌는지 빠르게 판별	요구사항을 하나하나 검증
크기	TC 7개 / 체크 46개	테스트 15개 (원래 요구사항 58건 흡수)
언제	RTL을 고칠 때마다	검증 단계에서
상태	TB에 구현되어 있음, 46/46 PASS	TB 보강(B1 ~ B6)이 필요한 테스트가 있음

> ⚠️ **Sanity가 통과했다고 해서 연결된 SOR 테스트가 통과한 것이 아닙니다.** SOR 테스트는 같은 동작을 **더 많이** 봅니다. 예를 들어 `SAN-004`는 SOR T-03의 일부(데이터·guard·완료 표시)만 봅니다. descriptor 읽기 모양(`ARLEN = 5`)은 Sanity에서 **로그에 찍히기만 하고 판정하지 않습니다.** SOR v2.0.0도 T-01 ~ T-04, T-09를 "Sanity를 다시 확인하는 단계"로 두고 따로 돌립니다.

1.5 처음 보는 말 풀이

말	쉬운 뜻
DUT	검증하는 회로. 여기서는 `sgdma_top`
RTL	회로를 글로 적은 설계 코드 (Verilog)
TB (testbench)	DUT에 신호를 넣고 결과를 검사하는 시뮬레이션 환경. 여기서는 `tb_sg_dma.sv`
TC	테스트 케이스. Sanity는 `SAN-001` ~ `SAN-007` 7개
체크	TC 안의 비교 하나. "읽은 값 = 기대값인가"를 한 번 보는 것. 모두 46개
회귀 (regression)	코드를 고친 뒤, 전에 되던 것이 여전히 되는지 다시 돌려 보는 것
게이트	통과해야 다음 단계로 갈 수 있는 관문
SKIP	그 TC를 아예 실행하지 않고 건너뜀
watchdog	정해진 시간이 지나면 시뮬레이션을 강제로 끝내는 감시 장치
hang	멈춰서 끝나지 않는 상태
AXI4-Lite	CPU가 레지스터를 읽고 쓰는 간단한 버스 (`s_axi` 포트)

말	쉬운 뜻
AXI	칩 안에서 데이터를 주고받는 표준 버스 규칙. DMA는 이걸로 메모리를 읽고 씀
AR / AW	읽기 주소 통로 / 쓰기 주소 통로
`BRESP` / `RRESP`	쓰기 / 읽기 결과 응답. `OKAY`(성공)이면 0
beat	버스의 한 박자. 이 DUT는 한 박자에 8바이트
burst	주소 한 번으로 여러 박자를 묶어 보내는 덩어리. 이 DUT는 최대 128바이트
descriptor	전송 1건의 할 일 메모 (48바이트). 소스·목적지·길이·다음 메모 위치가 적혀 있음
체인	descriptor가 "다음 메모 주소"로 줄줄이 이어진 것
gather	흩어진 소스 여러 곳을 목적지 한 곳에 이어 붙여 쓰기
scatter	소스 한 곳을 목적지 여러 곳에 나눠 쓰기 (Sanity는 **안 봄**)
prefetch	다음 descriptor를 미리 읽어 두기
W1C	1을 쓰면 지워지는 비트
SOT / EOT	전송 접수(Start Of Transfer) / 전송 완료(End Of Transfer) 이벤트. `IRQ_SOURCE`의 비트0 / 비트1
마스크	알림(인터럽트)을 막는 설정. `IRQ_MASK` 비트가 1이면 막힘
tail (guard 영역)	목적지 끝 바로 뒤 16바이트. 써지면 안 되는 감시 구역
stuck-at-1	고장으로 비트가 항상 1에 붙어 있는 상태
빌드 구성 레지스터	합성 파라미터를 그대로 보여 주는 고정값 레지스터 (`INTERFACE_DESCRIPTION_1/2`, `PERIPHERAL_ID`)
구현 특성 체크	ADI가 약속한 동작이 아니라, 지금 RTL이 우연히 그렇게 동작하는 것을 확인하는 체크. 이 문서에서는 `DESCRIPTOR_ID` 완료 후 값 2개 (3.8)
근거 등급 M / R	SFR의 표기. M = 실측으로 확인, R = RTL을 읽고 만든 값(아직 측정 안 함)

1.6 v1.0.0 대비 바뀐 점

TB는 바뀌지 않았습니다. 체크 46개, TC별 체크 수, 실행 순서, 기대값, 판정 규칙, 실측 로그(부록 A), 실행 명령은 v1.0.0과 같습니다. 바뀐 것은 문서뿐입니다.

#	바뀐 점	위치	근거
1	SOR 연결을 원래 61건 ID 기준에서 **SOR v2.0.0 테스트 번호(T-01 ~ T-15) 기준**으로 다시 매김. 원래 ID는 괄호로 남김	2.1, 3장 각 TC, 5.4	통합 문서 7장 #1
2	`DESCRIPTOR_ID` 완료 후 값 체크(`0xB2`, `0xC5`)를 **구현 특성 체크**로 표시하고 해석 규칙을 붙임. TB 동작은 그대로	3.6, 3.7, 3.8, 4.3, 부록 B	통합 문서 7장 #2, SOR v2.0.0 2.1절
3	"Sanity가 확인하지 않는 것" 목록을 SOR v2.0.0의 15개 테스트 기준으로 갱신 (scatter → T-05, 바이트 단위 길이 → T-07, 에러 → T-15 …)	5.2, 5.3, 5.4	통합 문서 7장 #3
4	디버그 레지스터(`0x434` ~ `0x444`) 비교 금지, `0x450` 읽기 금지 규칙을 한 곳(5.5)에 모아 강조. SOR v2.0.0의 "관찰 기록" 규칙과 연결	5.5, 7.2	SOR v2.0.0 2.1절
5	문서 참조를 v2.0.0으로 바꿈 (`SGDMA-SOR-rel-v2.0.0`, `SGDMA-SFR-rel-v2.0.0`)	머리말, 7.3	—
6	처음 보는 사람도 읽을 수 있게 다시 씀: 비유(1.1), 용어 풀이(1.5), 체크마다 "틀리면 무슨 뜻인지" 열 추가	1장, 3장	—

> 참고: **Sanity에는 `0xA0` 체크가 없습니다.** `SAN-004`는 `DESCRIPTOR_ID`를 읽지 않습니다. `0xA0`은 `SAN-004` descriptor의 `id`로만 쓰이고, 완료 후 값 `0xA0` 확인은 SOR T-03(참고 체크)에만 있습니다.

2. 구성

2.1 TC 목록

TC	이름	체크	계층	목적	주로 연결되는 SOR 테스트
`SAN-001`	레지스터 신원 확인 (Register identity)	5	레지스터	버스·디코딩이 살아 있는지, 빌드 구성 확인	T-01
`SAN-002`	스크래치 읽기·쓰기 (Scratch R/W)	3	레지스터	쓰기 경로가 살아 있는지	T-01
`SAN-003`	리셋 기본값 (Reset defaults)	10	레지스터	리셋 직후 값	T-09 (`IRQ_MASK` 만)
`SAN-007`	일부러 꺼 둔 비트 (Parameter-masked bits)	11	레지스터	**오판 방지** — 일부러 꺼 둔 비트를 버그로 보지 않게	T-02
`SAN-004`	descriptor 1장 복사 (Single descriptor copy)	4	전송	SG 기본 동작	T-03
`SAN-005`	3장 체인 모으기 (3-descriptor chain, gather)	6	전송	**체인 따라가기** (SG만의 기능)	T-04
`SAN-006`	인터럽트와 상태 (IRQ & status)	7	전송	인터럽트 경로	T-09
	합계	**46**			

TC별 상세 연결(부분 겹침 포함)은 5.4에 있습니다.

2.2 실행 순서

```
SAN-001 -> SAN-002 -> SAN-003 -> SAN-007 -> SAN-004 -> SAN-005 -> SAN-006
|------------ 레지스터 계층 ------------|  |-------- 전송 계층 --------|
```

> ⚠️ **순서에 의미가 있습니다. 임의로 바꾸면 안 됩니다.**

규칙	이유
레지스터 계층을 먼저	버스가 죽은 상태에서 전송을 돌리면 **원인을 나눌 수 없습니다.** "전송이 안 된다"와 "레지스터를 못 쓴다"가 구분되지 않습니다
`SAN-003` 은 `SAN-007` 보다 앞	`SAN-007` 이 `0xFFFF_FFFF` 를 씁니다. 뒤에 두면 리셋값을 볼 수 없습니다
`SAN-002` 는 `SAN-003` 보다 앞이어도 괜찮음	`SCRATCH` (0x008)는 `SAN-003` 의 확인 대상이 아닙니다
`SAN-007` 은 **되돌린 뒤 끝냄**	`CONTROL` 을 0으로 되돌리지 않으면 `SAN-004` 가 `ENABLE` 이 켜진 상태로 시작합니다

전송 계층에 들어가기 전에 공통 설정을 합니다.

```
        axil_write(REG_IRQ_MASK, 32'h1);   // SOT 마스크, EOT 허용
        axil_write(REG_CONTROL,  32'h5);   // ENABLE | HWDESC
```

`IRQ_MASK = 0x1` 은 "접수(SOT) 알림은 막고, 완료(EOT) 알림만 받겠다"는 뜻입니다. `CONTROL = 0x5` 는 "켜기(ENABLE) + SG 모드(HWDESC)"입니다. SOR v2.0.0 1.2절 "표준 순서"의 2·3번과 같습니다.

2.3 판정 규칙

상황	동작
체크 1건 통과	`[ ok ]` 출력, TC pass 카운터 +1
체크 1건 실패	`[FAIL]` + 기대값·실측값 출력, TC fail 카운터 +1
TC 안에서 fail ≥ 1	그 TC는 **FAIL**. 단, **이후 TC는 계속 실행**
`VERSION` 또는 `IDENTIFICATION` 불일치	**나머지 6개 TC를 전부 SKIP**하고 바로 끝냄
FAIL 또는 SKIP이 1건이라도 있음	`*** SANITY FAILED ***` + `$fatal(1)`
전부 통과	`*** SANITY PASSED ***` + `$finish`

중단 조건을 `VERSION` / `IDENTIFICATION` 둘로 좁힌 것은 설계 판단입니다.

처음 구현은 `SAN-001`의 체크가 하나라도 틀리면 나머지를 전부 SKIP했습니다. 그런데 실제로 `INTERFACE_DESCRIPTION_2` 기대값이 틀려서, **정상 동작하는 RTL인데 6개 TC가 전부 SKIP**된 일이 있었습니다 (부록 C).

`INTERFACE_DESCRIPTION_1/2`와 `PERIPHERAL_ID`는 **빌드 구성 레지스터**입니다. 값이 틀리면 **기대값이 틀린 것이지 버스가 죽은 것이 아닙니다.** 그래서 `bus_ok` 플래그를 따로 두고, `VERSION`과 `IDENTIFICATION` 둘만 게이트로 씁니다.

```
        bit bus_ok = 1'b1;   // cleared only when the bus itself looks dead
        ...
        axil_read(REG_VERSION, rd);
        check_eq("VERSION", rd, EXP_VERSION);
        if (rd !== EXP_VERSION) bus_ok = 1'b0;
```

구성 레지스터가 틀리면 아래 안내 문구를 출력합니다.

```
note : build-configuration register mismatch - confirm the
       expected value before treating it as an RTL defect.
```

> ⚠️ **판정 규칙에는 체크별 예외가 없습니다.** 3.8의 구현 특성 체크 2개도 다른 체크와 똑같이 집계됩니다. 즉 지금 TB에서는 이 체크가 틀려도 `*** SANITY FAILED ***`가 됩니다. **해석**만 다르게 합니다.

---

3. TC 상세

각 TC는 **목적 → 절차·구성 → 체크 → 주의 → SOR 연결** 순서입니다. 체크 표의 "틀리면" 열은 그 체크가 FAIL했을 때 **먼저 의심할 것**입니다. 모든 기대값의 상세 근거는 SFR 문서에 있습니다 (부록 B).

3.1 `SAN-001` — 레지스터 신원 확인 (Register identity, 5 checks)

목적: AXI4-Lite 버스와 레지스터 주소 해석(디코딩)이 살아 있는지 봅니다. 그리고 **의도한 파라미터로 합성되었는지** 확인합니다.

#	체크	기대값	SFR	틀리면
1	`VERSION` (0x000)	`0x0004_0565`	[4.1.1]	버스나 디코딩이 깨짐. **나머지 TC 전부 SKIP**
2	`IDENTIFICATION` (0x00C)	`0x444D_4143` ("DMAC")	[4.1.4]	위와 같음. **나머지 TC 전부 SKIP**

#	체크	기대값	SFR	틀리면
3	`PERIPHERAL_ID` (0x004)	`0x0000_0000`	[4.1.2]	`ID` 파라미터가 바뀌었거나 기대값이 틀림. 버스 고장보다 기대값을 먼저 의심
4	`INTERFACE_DESCRIPTION_1` (0x010)	`0x3807_0303`	[4.1.5]	다른 파라미터로 합성됨, 또는 파라미터를 바꾸고 기대값을 안 고침
5	`INTERFACE_DESCRIPTION_2` (0x014)	`0x0000_0000`	[4.1.6]	`AXI_AXCACHE` / `AXI_AXPROT` 설정이 바뀜. 기대값부터 확인

체크 4가 가장 많은 정보를 담고 있습니다. `INTERFACE_DESCRIPTION_1`은 빌드 파라미터를 묶어 놓은 상수 레지스터입니다. 이 한 값이 맞으면 `DMA_TYPE_SRC/DEST`(둘 다 AXI-MM), `BYTES_PER_BEAT`(8), `MAX_BYTES_PER_BURST`(128), `MAX_NUM_FRAMES`가 전부 의도대로임이 증명됩니다.

```
 31    27 26 25 24 23  20 19  16 15 14 13 12 11   8 7  6 5  4 3    0
+--------+--+--+--+------+------+-----+-----+------+----+-----+------+
|MAX_NUM_|2D|EX|AR| rsvd |BPBRST|rsvd |TYPE | BPB  |rsvd|TYPE | BPB  |
|FRAMES-1|TL|SY|UN|      |_WIDTH|     | SRC | SRC  |    |DEST | DEST |
+--------+--+--+--+------+------+-----+-----+------+----+-----+------+
    7     0  0  0    0      7      0     0     3     0     0     3    -> 0x3807_0303
```

> ⚠️ **체크 5의 기대값은 과거에 틀렸던 값입니다.** `0x30`이라고 적은 문서가 두 건 있었습니다. 둘 다 `axi_dmac_regmap.v`의 **로컬 기본값** `AXI_AXCACHE = 4'b0011`을 보고 계산한 것이었습니다. 실제로는 상위 모듈 `axi_dmac`이 넘기는 값이 적용됩니다. 2026-10-06부터 `sgdma_top`이 `4'b0000`을 **명시**하므로 이 혼동은 더 생기지 않습니다. TB 주석에도 경고를 남겼습니다.

SOR 연결

- **T-01** 레지스터 버스와 기본 구성 확인 (`REG-001`, `REG-002`, `REG-004`, `REG-005`, `REG-020`) — 같은 5개 값을 봅니다. `VERSION`/`IDENTIFICATION`이 틀리면 나머지를 건너뛰는 규칙도 같습니다.
- **T-14** 느린 버스와 AXI 속성 (`AXI-007`) — 체크 5는 **레지스터 값**만 봅니다. 세 포트가 실제로 내보내는 `AxCACHE`/`AxPROT` 신호는 Sanity가 보지 않습니다. 그것은 T-14가 봅니다.

3.2 `SAN-002` — 스크래치 읽기·쓰기 (Scratch R/W, 3 checks)

목적: 쓰기 경로가 살아 있는지 봅니다. `SCRATCH`(0x008)는 기능이 없는 레지스터라, 써도 부작용이 없습니다.

#	쓰는 값	읽기 기대	틀리면
1	`0xA5A5_5A5A`	같은 값	쓰기 경로 고장, 또는 일부 비트가 고착
2	`0x5A5A_A5A5`	같은 값	위와 같음. 1과 함께 보면 어느 비트가 고착인지 보임
3	`0x0000_0000`	같은 값	0을 못 씀 (stuck-at-1). 다음 TC를 위한 원복도 실패

패턴을 반전해 두 번 쓰는 것이 핵심입니다. 한 패턴만 쓰면 일부 비트가 1로 붙어 있어도(stuck-at-1) 통과합니다. `0xA5A5_5A5A`와 그 반전을 모두 확인해야 32비트 전체가 0↔1 양방향으로 동작함이 보장됩니다.

세 번째로 0을 쓰는 것은 **다음 TC를 위한 원복**입니다.

SOR 연결

- **T-01** (`REG-003`) — 같은 세 값을 쓰고 읽습니다.
- `WSTRB`(일부 바이트만 쓰기) 동작은 Sanity가 보지 않습니다 → **T-02** (`REG-023`).

3.3 `SAN-003` — 리셋 기본값 (Reset defaults, 10 checks)

목적: HW 리셋이 풀린 직후의 레지스터 기본값을 확인합니다.

#	레지스터	기대값	비고	틀리면
1	`CONTROL` (0x400)	`0x0000_0000`	**`HWDESC`도 0**	리셋 시퀀스 또는 초기값 변경
2	`IRQ_MASK` (0x080)	**`0x0000_0003`**	**0이 아님** — 둘 다 막힘	초기값 변경. 드라이버의 "unmask" 전제가 깨짐
3	`IRQ_PENDING` (0x084)	`0x0000_0000`		리셋 직후 알림이 떠 있음
4	`IRQ_SOURCE` (0x088)	`0x0000_0000`		리셋 직후 이벤트가 기록되어 있음
5	`FLAGS` (0x40C)	**`0x0000_0002`**	**0이 아님** — `TLAST` 기본 1	초기값 변경
6	`TRANSFER_DONE` (0x428)	`0x0000_0000`		리셋 직후 완료 표시가 남아 있음
7	`TRANSFER_ID` (0x404)	`0x0000_0000`		전송 번호 초기값 변경
8	`DEST_ADDRESS` (0x410)	`0x0000_0000`		리셋이 주소 레지스터에 안 걸림
9	`SRC_ADDRESS` (0x414)	`0x0000_0000`		위와 같음
10	`SG_ADDRESS` (0x47C)	`0x0000_0000`		위와 같음

> ⚠️ **0이 아닌 리셋값이 둘 있습니다 — `IRQ_MASK = 0x3`, `FLAGS = 0x2`.** "리셋 후 전 레지스터 0"을 가정한 테스트는 **이 둘에서만** FAIL합니다. 이 IP를 처음 다루는 사람이 가장 많이 걸리는 곳입니다.

`CONTROL.HWDESC` 리셋값이 0인 점을 특히 기억하십시오. "SG 모드를 항상 쓴다"는 것은 **소프트웨어 운용 전제**이지 하드웨어 기본값이 아닙니다. 드라이버가 `CONTROL`에 `0x5`(ENABLE | HWDESC)를 직접 써야 SG가 동작합니다.

이 TC는 10개만 봅니다. SFR 부록 A에는 약 30개가 있습니다. 나머지 목록은 5.2에 있습니다.

SOR 연결

- **T-09** 알림 막기와 지우기 (`IRQ-003`) — 1단계에서 리셋 직후 `IRQ_MASK = 0x0000_0003`을 다시 봅니다.
- 나머지 9개 리셋값은 **SOR v2.0.0에 대응 테스트가 없습니다.** 리셋 관련은 SOR에서 다루지 않기로 한 팀 결정 때문입니다 (5.3). 그래서 **이 리셋값들의 실측 근거는 이 TC뿐입니다.**

3.4 `SAN-007` — 일부러 꺼 둔 비트 (Parameter-masked bits, 11 checks)

목적은 버그를 찾는 것이 아니라 오판을 막는 것입니다.

파라미터로 **일부러 꺼 둔 비트**가 있습니다. 이걸 모르는 사람이 나중에 "RW 레지스터 쓰고 다시 읽기" 테스트를 만들면 **정상 동작을 버그로 보고**합니다. 기대값을 문서와 TB 양쪽에 고정해 그것을 막습니다.

#	동작	기대값	꺼진 이유	틀리면
1	`W(0x400, 0xFFFF_FFFF)` → `R`	`0x0000_0007`	`FRAMELOCK = 0` → `[3]` 마스크	`FRAMELOCK` 파라미터가 바뀜
2	`W(0x40C, 0xFFFF_FFFF)` → `R`	`0x0000_0006`	`CYCLIC = 0` → `[0]` 마스크	`CYCLIC` 파라미터가 바뀜
3	`W(0x410, 0xFFFF_FFFF)` → `R`	`0xFFFF_FFF8`	64b 버스 → 8B 정렬	데이터 폭 파라미터가 바뀜, 또는 정렬 마스크 로직 변경
4	`W(0x414, 0xFFFF_FFFF)` → `R`	`0xFFFF_FFF8`	위와 같음	위와 같음

#	동작	기대값	꺼진 이유	틀리면
5	`W(0x47C, 0xFFFF_FFFF)` → `R`	`0xFFFF_FFF8`	위와 같음	위와 같음
6	`W(0x490, 0xFFFF_FFFF)` → `R`	`0x0000_0000`	`ADDR_WIDTH = 32` → `HAS_ADDR_HIGH = 0`	주소 폭이 64로 바뀜 (7.1)
7	`W(0x494, 0xFFFF_FFFF)` → `R`	`0x0000_0000`	위와 같음	위와 같음
8	`W(0x4BC, 0xFFFF_FFFF)` → `R`	`0x0000_0000`	위와 같음	위와 같음
9	`R(0x41C)` `Y_LENGTH`	`0x0000_0000`	`DMA_2D_TRANSFER = 0`	2D 전송이 켜짐
10	`R(0x458)` `FRAMELOCK_CONFIG`	`0x0000_0000`	`FRAMELOCK = 0`	`FRAMELOCK` 파라미터가 바뀜
11	`R(0x0F0)`, `R(0x600)` 미매핑	둘 다 `0x0000_0000`	정의되지 않은 주소	**주소 디코드 오류**

체크 6~8의 뜻: `*_ADDRESS_HIGH` 레지스터는 "0을 돌려주는" 것이 아닙니다. `if (HAS_ADDR_HIGH)` 가드 때문에 **쓰기 로직 자체가 합성되지 않습니다.** 레지스터가 존재하지 않습니다.

체크 11이 중요한 이유: 이 IP는 **정의되지 않은 주소에도 에러 응답을 내지 않습니다.** `BRESP`/`RRESP`가 항상 `OKAY`입니다. 따라서 주소 디코드 오류를 잡으려면 **다시 읽은 값을 비교하는 것이 유일한 방법**입니다.

> ⚠️ **`0x450`(`PARTIAL_TRANSFER_ID`)은 일부러 읽지 않습니다.** 이 주소는 **읽기만 해도 내부 FIFO가 하나 빠집니다**(pop) (`up_tlf_rd = up_rreq && up_raddr == 'h114'`). 이 구성에서는 FIFO가 항상 비어 있어 실제 피해가 없습니다. 하지만 **"전 레지스터 덤프" 방식의 테스트를 만들면 안 되는 이유**이므로 원칙으로 고정했습니다. SFR 4.5.5 참조. 규칙 전체는 5.5에 있습니다.

이 TC는 건드린 레지스터를 전부 되돌린 뒤 끝납니다.

```
        axil_write(REG_CONTROL,      32'h0000_0000);   // 안 하면 SAN-004가 ENABLE=1로 시작
        axil_write(REG_FLAGS,        EXP_FLAGS_RESET);
        axil_write(REG_DEST_ADDRESS, 32'h0000_0000);
        axil_write(REG_SRC_ADDRESS,  32'h0000_0000);
        axil_write(REG_SG_ADDRESS,   32'h0000_0000);
```

SOR 연결

- **T-02** 일부러 꺼 둔 비트와 레지스터 접근 규칙 (`REG-021`, `REG-022`) — 같은 11개 동작을 봅니다. T-02는 여기에 더해 `*_STRIDE`·`FRAMELOCK_STRIDE` 읽기, 읽기 전용 레지스터 쓰기(`REG-019`), `WSTRB` 무시(`REG-023`)를 봅니다. 이것들은 Sanity에 없습니다.
- **T-13** 주소 정렬과 4 KB 경계 (`DESC-008`) — 일부만 겹칩니다. 체크 5는 `SG_ADDRESS` **레지스터**가 아래 3비트를 버리는 것만 봅니다. 실제 descriptor 읽기 주소가 맞춰지는지는 T-13이 봅니다.

3.5 `SAN-004` — descriptor 1장 복사 (Single descriptor copy, 4 checks)

목적: SG 기본 동작을 봅니다. descriptor 1장을 읽고 메모리→메모리 전송을 끝내는지 확인합니다.

구성

descriptor	`0x0000_1000` · `id = 0xA0` · `flags = 0b11` (LAST | EOT_IRQ)

전송	`0x0001_0000` → `0x0002_0000`, **256 byte**

#	체크	내용	틀리면
1	`irq asserted`	EOT 인터럽트 발생 (타임아웃 50,000 사이클)	전송이 끝나지 않음, 또는 인터럽트 경로 고장 (4.3)
2	`data`	256 byte 전부 바이트 단위 일치	descriptor 해석(주소·길이) 또는 데이터 경로 오류
3	`tail`	목적지 끝 뒤 16 byte가 **쓰이지 않음**	**범위를 넘어 씀** — 길이 해석이 1 이상 큼
4	`TRANSFER_DONE bit set`	`[3:0]` 중 하나 이상 1	완료 표시 경로 고장

체크 3(tail)이 필요한 이유: 데이터가 맞는 것만으로는 부족합니다. **범위를 넘어 쓰는 버그**는 데이터 비교로 잡히지 않습니다. 메모리 모델이 "쓰였는지"를 바이트 단위로 기록하므로(`mem_written()`), 목적지 뒤 16 byte가 한 번도 쓰이지 않았음을 확인합니다.

관측된 버스 동작 (실측)

```
[3195000]   SG AR  addr=0x00001000 beats=6  size=8B   <- descriptor fetch, 48 B
[3315000]  SRC AR  addr=0x00010000 beats=16 size=8B   <- 128 B
[3495000]  DST AW  addr=0x00020000 beats=16 size=8B
[3495000]  SRC AR  addr=0x00010080 beats=16 size=8B   <- 128 B (분할)
[3705000]  DST AW  addr=0x00020080 beats=16 size=8B
```

- descriptor 읽기가 **6 beat × 64bit = 48 byte 단일 INCR burst**로 나갑니다 (`ARLEN=5`, `ARSIZE=3`).
- 256 byte 전송이 `MAX_BYTES_PER_BURST = 128`에 따라 **2개 burst로 나뉩니다.**

> ⚠️ 위 버스 동작은 **로그에 찍힐 뿐 판정하지 않습니다.** 46개 체크에 들어 있지 않습니다. 판정은 SOR 쪽에서 합니다: descriptor 읽기 모양은 T-03(`AXI-001`, `DESC-001`), 128 byte 분할은 T-07(`FUNC-006`), "소스를 다 읽은 뒤 쓰기"는 T-07(`FUNC-007`).

SOR 연결

- **T-03** descriptor 1장으로 복사하기 (`REG-007`, `REG-015`, `DESC-001`, `DESC-002`, `DESC-003`, `DESC-007`, `FUNC-001`, `FUNC-009`, `AXI-001`) — 같은 descriptor·주소·길이를 씁니다. T-03은 버스 요청 모양, `SG_ADDRESS` 다시 읽기, `DESCRIPTOR_ID = 0xA0`(참고 체크)까지 봅니다. Sanity는 `DESCRIPTOR_ID`를 이 TC에서 읽지 않습니다.
- **T-07** 1바이트 단위 길이 전송 (`FUNC-006`, `FUNC-007`) — 위 로그의 3,315 ns → 3,495 ns를 참고 실측으로 씁니다.
- **T-10** 출발 버튼 규칙과 연속 예약 (`REG-013`) — 체크 4는 "완료하면 비트가 선다"만 봅니다. "새로 제출하면 지워진다"는 T-10이 봅니다.

3.6 `SAN-005` — 3장 체인 모으기 (3-descriptor chain, gather, 6 checks)

SG의 핵심 기능을 보는 TC입니다. 체인 따라가기(`next_sg_addr` 따라가기)가 안 되면 SG가 아닙니다.

구성 — 흩어진 소스 3곳을 이어진 목적지로 모읍니다(gather).

descriptor	주소	id	flags	소스	목적지	길이
0	`0x0000_1100`	`0xB0`	`0b00`	`0x0001_1000`	`0x0002_1000`	128
1	`0x0000_1140`	`0xB1`	`0b00`	`0x0001_3000`	`0x0002_1080`	256
2	`0x0000_1180`	`0xB2`	**`0b11`**	`0x0001_5000`	`0x0002_1180`	64

마지막 descriptor에만 `flags = 0b11`(LAST | EOT_IRQ)을 둡니다. 이렇게 하면 `TRANSFER_ID`와 `TRANSFER_DONE`이 1:1로 맞고, 인터럽트도 체인당 1번입니다. 모든 descriptor에 `flags[1]`을 세우면 두 카운터가 어긋납니다(SFR 6.2).

#	체크	내용	틀리면
1	`irq asserted`	체인 전체가 끝난 뒤 1번	체인을 끝까지 못 따라감, 또는 LAST에서 안 멈춤
2~4	`seg0` / `seg1` / `seg2`	조각마다 바이트 단위 일치	틀린 조각 번호의 descriptor 해석 또는 "다음 장 주소" 따라가기 오류
5	`tail`	448 byte 뒤 16 byte가 안 쓰임	마지막 조각이 넘쳐 씀
6	`DESCRIPTOR_ID = last of chain`	`0x0000_00B2`	**구현 특성 체크.** RTL 버그가 아니라 "구현이 바뀌었다"는 신호로 먼저 봄 (3.8)

관측된 prefetch 동작 (실측)

```
[4215000]   SG AR  addr=0x00001100    <- desc 0
[4295000]   SG AR  addr=0x00001140    <- desc 1
[4335000]  SRC AR  addr=0x00011000    <- 첫 데이터 전송 시작
[4375000]   SG AR  addr=0x00001180    <- desc 2
```

descriptor 3장이 첫 데이터 전송이 끝나기 전에 모두 읽힙니다. descriptor 읽기와 데이터 전송이 겹쳐서(파이프라인으로) 진행된다는 직접 증거입니다. 이 동작도 **로그로만 남고 판정하지 않습니다.** 판정은 T-04(`DESC-009`)가 합니다.

SOR 연결

- **T-04** 체인 따라가기와 모으기 (`DESC-004`, `DESC-009`, `FUNC-002`) — 같은 3장 체인을 씁니다. T-04는 descriptor 읽기 순서·횟수, prefetch, `irq` 횟수, `TRANSFER_ID` 증가를 판정합니다.
- **T-06** 체인 길이 1장 ~ 32장 (`FUNC-004`) — Sanity는 **3장만** 봅니다.
- **T-08** 중간 알림과 descriptor 번호 (`DESC-005`, `DESC-006`, `REG-016`) — Sanity는 "마지막 장에만 알림" 한쪽만 봅니다. 체크 6은 T-08의 `DESCRIPTOR_ID` 관측과 겹치며, SOR에서도 **참고 체크**입니다.

3.7 `SAN-006` — 인터럽트와 상태 (IRQ & status, 7 checks)

목적: 인터럽트 경로 전체를 봅니다 — 이벤트 기록 → 마스크로 거르기 → `irq` 출력 → W1C로 지우기.

구성: descriptor 1장 (`id = 0xC5`, 64 byte), `IRQ_MASK = 0x1` (SOT 막음, EOT 허용)

#	체크	기대	확인 내용	틀리면
1	`IRQ_MASK = 0x1`	`0x0000_0001`	앞서 쓴 마스크가 유지됨	마스크 레지스터 쓰기·유지 고장
2	`irq asserted`	—	전송이 끝나면 `irq`가 올라감	전송 미완료, 또는 EOT가 `irq`까지 안 옴
3	`IRQ_PENDING[1] set`	`[1] = 1`	EOT가 pending으로 전달됨	마스크 거르기 로직 오류
4	`IRQ_PENDING cleared`	`0x0000_0000`	W1C 뒤	W1C가 안 먹음
5	**`IRQ_SOURCE[0] survives W1C`**	**`0x0000_0001`**	**막아 둔 SOT가 SOURCE에는 남음**	마스크가 이벤트 기록까지 막도록 동작이 바뀜. 또는 W1C가 쓴 비트 외의 비트까지 지움
6	`irq de-asserted after W1C`	`irq = 0`	레벨 신호가 내려감	지운 뒤에도 `irq`가 남음 — 인터럽트 폭주 위험
7	`DESCRIPTOR_ID`	`0x0000_00C5`	완료 후 값	**구현 특성 체크.** RTL 버그가 아니라 "구현이 바뀌었다"는 신호로 먼저 봄 (3.8)

체크 5가 이 TC에서 가장 값진 체크입니다.

`IRQ_MASK`는 **이벤트 기록을 막지 않습니다.** 막아 둔 SOT 이벤트도 `IRQ_SOURCE[0]`에 기록됩니다. `IRQ_PENDING`에만 안 보일 뿐입니다. 그래서 W1C로 `[1]`만 지운 뒤 `IRQ_SOURCE`를 읽으면 `0x1`이 남아 있습니다.

실무에서는 이런 뜻입니다.

> ⚠️ **마스크를 푸는 순간 과거 이벤트가 튀어나옵니다.** unmask 전에 `IRQ_PENDING`에 `0x3`을 써서 비우지 않으면, unmask 직후 의도하지 않은 인터럽트가 올라옵니다. 드라이버와 테스트 모두 이 순서를 지켜야 합니다.

`IRQ_PENDING`에 쓰면 실제로 지워지는 것은 `IRQ_SOURCE`입니다. 이름과 실제가 어긋나는 곳이라 시나리오를 쓸 때 반드시 구분해야 합니다(SFR 4.2.2).

SOR 연결

- **T-09** 알림 막기와 지우기 (`IRQ-002`, `IRQ-003`, `IRQ-004`, `IRQ-005`) — Sanity는 마스크 `0x1` **한 가지**만 봅니다. T-09는 `0x3`, `0x2`, `0x1`, `0x0` 네 가지와 "비우지 않고 unmask" 경우까지 봅니다.
- **T-08** 중간 알림과 descriptor 번호 (`IRQ-001`, `REG-016`, `DESC-006`) — 체크 2는 "완료 이벤트가 기록되고 알림이 온다"의 가장 단순한 경우입니다. 체크 7은 T-08의 `DESCRIPTOR_ID` 관측과 겹치며, SOR에서도 **참고 체크**입니다.

3.8 구현 특성 체크 — `DESCRIPTOR_ID` 완료 후 값

이 절은 두 체크에 공통으로 적용됩니다.

체크	기대값	부록 B 번호
`SAN-005 #6 DESCRIPTOR_ID = last of chain`	`0x0000_00B2`	39
`SAN-006 #7 DESCRIPTOR_ID`	`0x0000_00C5`	46

무엇이 "구현 특성"인가

전송이 끝난 뒤 `DESCRIPTOR_ID`가 마지막 descriptor의 id를 계속 보여 주는 것은 `util_axis_fifo`의 구현 특성입니다. ADI가 문서로 약속한 동작이 아닙니다.

`dmac_sg`는 목적지 응답이 올 때마다 완료 추적 FIFO에서 하나를 꺼냅니다(pop). FIFO가 비면 출력이 갱신되지 않아 마지막 값이 남습니다. 선행 문서에는 "이전 전송의 stale 값(남아 있던 옛 값)이 보일 수 있다"는 경고가 있었습니다. **`SAN-005`가 그것을 반증했습니다.** 앞선 `SAN-004`가 남긴 `0xA0`이 아니라 자기 체인의 `0xB2`가 나왔습니다.

해석 규칙 (SOR v2.0.0 2.1절과 같음)

SOR v2.0.0은 판정을 네 종류로 나눕니다: **판정 / 참고 체크 / 관찰 기록 / 관측 후 확정**. 그리고 "`DESCRIPTOR_ID`의 전송이 **끝난 뒤** 값은 항상 **참고 체크**"라고 정했습니다. 이 두 Sanity 체크는 그 규칙의 대상입니다.

> ⚠️ **이 체크가 FAIL하면, RTL 버그가 아니라 "구현이 바뀌었다"는 신호로 먼저 봅니다.**
>
> 1. 먼저 `dmac_sg.v`와 `util_axis_fifo`가 바뀌었는지 확인합니다.
> 2. 바뀌었고 다른 44개 체크가 모두 통과했다면, 기능 고장이 아니라 **구현 특성의 변화**입니다. 기대값과 이 문서를 고칩니다.
> 3. 바뀐 것이 없는데 FAIL하면, 그때 RTL 문제로 조사합니다.

> ⚠️ **드라이버는 이 값을 완료 판정 근거로 쓰면 안 됩니다.** 완료는 EOT 인터럽트나 `TRANSFER_DONE`으로 확인합니다. 이 레지스터는 디버깅·추적용입니다 (SFR 4.6.2).

지금 TB에서는 어떻게 처리되나

2.3 판정 규칙에는 체크별 예외가 없습니다. 그래서 **지금 TB에서는 이 두 체크도 다른 44개와 똑같이 게이트 판정에 들어갑니다.** 틀리면 그 TC가 FAIL이 되고, 전체 결과는 `*** SANITY FAILED ***` + `$fatal(1)` 입니다. 문서가 바꾸는 것은 **FAIL을 읽는 방법**뿐입니다.

권장 (TB 변경 아님): 향후 TB에서 이 두 체크를 **경고(WARN)로 낮추어** 게이트 판정에서 빼는 것을 권장합니다. 그러면 구현이 바뀌었을 때 회귀 전체가 멈추지 않습니다. 바꿀 경우 체크 수 집계와 요약 형식이 달라질 수 있으므로, 7.2 규칙대로 TB 헤더·이 문서·부록 B를 함께 고칩니다.

---

4. 실행 방법

4.1 기본 실행

```
cd xsim
source sourceme          # Vivado 환경 (설정되어 있으면 생략 가능)
make clean && make sim
```

타깃	동작
`make sim`	컴파일 + elaborate + 배치 실행. **회귀 게이트**
`make wave`	전 신호를 `.wdb`로 덤프하며 실행
`make view`	`make wave`가 만든 `.wdb`를 GUI로 열기
`make gui`	스냅샷을 GUI로 열기 (신호를 직접 추가 후 `run all`)
`make clean`	시뮬레이션 산출물 삭제

> ⚠️ **`make clean`을 생략하지 마십시오.** `xelab`은 소스가 아니라 **컴파일된 라이브러리**(`xsim.dir/work/`)를 읽습니다. RTL을 고친 뒤 `make sim`만 돌리면 **이전 코드가 그대로 돌 수 있습니다.** 모듈명을 바꿨을 때 특히 헷갈립니다.

4.2 판정

```
SANITY: 7 PASS / 0 FAIL / 0 SKIP     (46/46 checks)
*** SANITY PASSED ***
```

종료	의미
`*** SANITY PASSED ***` + `$finish`	통과. 종료 코드 0
`*** SANITY FAILED ***` + `$fatal(1)`	**실패.** 종료 코드 비정상
`GLOBAL TIMEOUT`	핸드셰이크가 멈춤. `$fatal(1)`

스크립트에서는 종료 코드만 보면 됩니다. 로그를 파싱할 필요가 없습니다.

4.3 실패했을 때

증상	먼저 볼 것
`SAN-001`의 `VERSION`/`IDENTIFICATION` FAIL	**버스 또는 디코딩이 깨짐.** 나머지는 SKIP되므로 이것부터
구성 레지스터(`SAN-001` 체크 3·4·5) FAIL	**기대값이 틀렸을 가능성이 먼저**입니다. 파라미터를 바꿨다면 7.1
`SAN-003` FAIL	리셋 시퀀스 또는 레지스터 초기값
`SAN-007` FAIL	파라미터가 바뀌었는지 확인. 기능을 바꾼 것이면 기대값을 같이 고쳐야 함
`irq asserted` 타임아웃	전송이 끝나지 않음. `make wave`로 `m_sg_axi` / `m_src_axi` / `m_dest_axi` 확인
`data` mismatch	처음 4건의 주소·값이 출력됩니다. 주소 패턴을 보면 정렬 문제인지 길이 문제인지 구분됩니다
`DESCRIPTOR_ID` FAIL (`SAN-005` #6, `SAN-006` #7)	**구현 특성 체크입니다.** RTL 버그로 단정하기 전에 `dmac_sg.v` / `util_axis_fifo`가 바뀌었는지 먼저 확인 (3.8)
`GLOBAL TIMEOUT`	어느 TC에서 멈췄는지 바로 앞의 출력으로 확인

5. 커버 범위와 한계

> ⚠️ **이 문서에서 가장 중요한 장입니다.** SOR v2.0.0에는 리셋값 전용 테스트가 없습니다 (T-01, T-09가 리셋 직후 값 일부만 읽음). 그래서 **리셋값의 실측 근거는 여전히 이 TB**입니다. 무엇을 확인했는지만큼, **무엇을 확인하지 않았는지**가 중요합니다.

5.1 커버하는 것

영역	내용	TC	SOR v2.0.0에서 더 보는 곳
버스 생존	AXI4-Lite 읽기·쓰기	SAN-001, 002	T-01
빌드 구성	파라미터가 의도대로 합성됨	SAN-001	T-01
리셋값	**10개 레지스터**	SAN-003	T-09 (`IRQ_MASK`만)
꺼 둔 비트	파라미터 마스킹 8종 + 미매핑 주소	SAN-007	T-02
SG 기본 동작	descriptor 읽기 → MM→MM 전송	SAN-004	T-03
SG 체인 따라가기	`next_sg_addr` 3단 + prefetch(관측)	**SAN-005**	T-04, T-06
descriptor 형식	`flags` · `id` · 주소 · `x_len` 해석	SAN-004, 005	T-03, T-04
burst 분할	128 B 단위 (**관측만, 판정 안 함**)	SAN-004	T-07
인터럽트 경로	기록 → 마스크 → pending → W1C → 하강 (마스크 `0x1`만)	SAN-006	T-09
범위 넘어 쓰기	목적지 뒤 16 byte 미기입	SAN-004, 005	모든 전송 테스트 (공통 조건 C3)

5.2 커버하지 않는 것 — 레지스터 리셋값

`SAN-003`이 보지 않는 리셋값입니다. 오른쪽 열은 SOR v2.0.0이 그 레지스터를 **어떤 형태로라도** 읽는지입니다. "리셋 직후 값"을 판정하는 SOR 테스트는 없습니다.

레지스터	이유	SOR v2.0.0에서
`0x408 TRANSFER_SUBMIT`	리셋값 체크 없음	T-10이 `ENABLE=0`에서 쓴 뒤 다시 읽음 (리셋값 측정은 아님)
`0x418 X_LENGTH`	**upstream 문서는 `7`, RTL 분석은 `0`으로 어긋남 — 미측정**	**어디에도 없음**
`0x420` / `0x424 *_STRIDE`	체크 없음 (꺼진 레지스터)	T-02가 값 `0`을 처음 측정 (리셋 직후 시점은 아님)
`0x42C ACTIVE_TRANSFER_ID`	체크 없음	T-10, T-11이 전송 중·끈 뒤 값을 판정
`0x430 STATUS`	체크 없음 (항상 0)	없음
`0x448 TRANSFER_PROGRESS`	체크 없음	T-12가 전송 중·끝난 뒤 값을 판정
`0x44C PARTIAL_TRANSFER_LENGTH`	체크 없음	없음
`0x454 DESCRIPTOR_ID`	**리셋 직후 값** 미확인 (전송 후 값은 SAN-005/006이 확인 — 구현 특성 체크)	T-08이 SG 모드를 끈 뒤 `0`을 참고 체크
`0x45C FRAMELOCK_STRIDE`	체크 없음	T-02가 값 `0`을 처음 측정 (리셋 직후 시점은 아님)
`0x498` / `0x49C CURRENT_*_HIGH`	체크 없음	없음
`0x450 PARTIAL_TRANSFER_ID`	**일부러 뺌** — 읽으면 FIFO가 빠짐	**읽기 금지** (T-02 6단계에 명시)
`0x434` ~ `0x444` 디버그군	**일부러 뺌** — 고정 리셋값이 없어 기대값을 정할 수 없음	**관찰 기록만** (T-11, T-12가 `DBG_STATUS`를 기록, 판정 안 함)

> ⚠️ **`0x418 X_LENGTH`는 주의가 필요합니다.** upstream 문서는 리셋값을 `2**clog2(버스폭/8)-1 = 7`로 적습니다. 이는 길이 정렬 제약이 있는 구성의 값입니다. 이 구성은 `DMA_LENGTH_ALIGN = 0`이라 RTL 분석상 `0`입니다. **둘 중 어느 쪽인지 측정되지 않았습니다.** SOR v2.0.0에도 이 값을 재는 테스트는 없습니다.

5.3 커버하지 않는 것 — 기능

Sanity 범위 밖인 기능과, 그것을 맡은 SOR v2.0.0 테스트입니다. `[B번호]`는 SOR v2.0.0 2.6절의 "testbench 보강 목록"입니다. 보강 전에는 그 판정이 보류됩니다.

영역	Sanity에서 빠진 내용	맡은 SOR 테스트 (원래 ID)
Scatter	이 TB는 **gather(N→1)만** 확인. **1→N scatter는 미검증**	**T-05** (`FUNC-003`)
체인 길이	3장만 확인. 1장·긴 체인 미확인	**T-06** (`FUNC-004`)
`TRANSFER_DONE` 4개 넘게 쌓기	미확인	SOR v2.0.0에도 판정 없음 (T-10은 4개 이내에서 확인하고, 번호 재사용 때 완료 표시가 지워지는 것만 6단계에서 봄)
전송 길이	64 / 128 / 256 byte만. **바이트 단위 비정렬 길이 미확인**	**T-07** (`FUNC-005`, `FUNC-006`, `FUNC-007`, `AXI-002`, `AXI-003`) [B1]
주소 정렬	8 byte 정렬만. 정렬 안 된 주소·descriptor 미확인	**T-13** (`FUNC-010`, `DESC-008`)
4 KiB 경계	burst가 경계를 넘는 경우 미확인	**T-13** (`AXI-004`) [B1]
연속 전송	큐잉, `TRANSFER_SUBMIT`이 저절로 0이 되는 타이밍	**T-10** (`REG-009` ~ `REG-014`, `FUNC-008`)
`ENABLE=0`에서 출발	조용히 무시되는지 미확인	**T-10** 1단계 (`REG-011`)

영역	Sanity에서 빠진 내용	맡은 SOR 테스트 (원래 ID)
descriptor별 IRQ	중간 descriptor에 `flags[1]`을 세운 경우	**T-08** (`DESC-005`, `IRQ-001`)
`DESCRIPTOR_ID` 전송 중 추적	완료 후 값만 봄 (구현 특성)	**T-08** (`DESC-006`, `REG-016`) [B6]
마스크 조합	`IRQ_MASK = 0x1`만	**T-09** (`IRQ-002` ~ `IRQ-005`)
`ENABLE` 끄기	끈 뒤 어떤 상태가 지워지고 어떤 설정이 남는지	**T-11** (`REG-006`, `REG-024`) [B6]
`PAUSE`	멈춘 자리부터 이어 가는지	**T-12** (`REG-008`, `REG-018`) [B6]
전송 중 관측	`TRANSFER_PROGRESS`	**T-12** (`REG-018`)
전송 중 관측	`CURRENT_*_ADDRESS`	**SOR에서도 제외** (`REG-017`) — 디버그 레지스터라 비교하지 않음
전송 중 관측	`DBG_STATUS`	**T-11**, **T-12** — **관찰 기록만** (`REG-024`)
읽기 전용 쓰기·`WSTRB`	읽기 전용 레지스터 쓰기, 일부 바이트 쓰기	**T-02** (`REG-019`, `REG-023`)
랜덤 지연	메모리 모델이 지연 없이 응답	**T-14** (`AXI-005`) [B4]
AXI 규칙 준수	assertion 없음. **`AxCACHE`가 실제로 `4'b0000`인지 미확인**	**T-14** (`AXI-007`) + 모든 테스트 공통 조건 C5 [B1]
여러 요청 동시 처리	미확인	**SOR에서도 제외** (`AXI-006`) — 지금 메모리 모델로는 시험 불가
에러 주입	`RRESP`/`BRESP` 에러, 잘못된 descriptor 주소·내용	**T-15** (`ERR-001` ~ `ERR-004`) [B3]
커버리지	측정 안 함	**SOR에서도 제외** (`COV-001`)

⚠️ **"전송 길이"(T-07)가 이 구성만의 가치가 모여 있는 곳입니다.** `DMA_LENGTH_ALIGN = 0`은 SRC·DEST가 **둘 다 AXI-MM**일 때만 성립합니다. **ADI 공식 testbench는 이 조합을 검증하지 않습니다**(두 config 모두 한쪽이 AXI-Stream). 바이트 단위 길이 지정은 **아무도 검증한 적이 없는 영역**입니다.

리셋 관련은 Sanity에도 SOR에도 추가하지 않기로 했습니다(2026-10-06 팀 결정). v1.0.0 SOR에서 [3-1 Reset / Clock]은 "Sanity Test에서 커버됨 · 해당 사항 없음"으로 정리되었고, `RST` 요구사항 6건은 삭제되었습니다. SOR v2.0.0도 이 결정을 그대로 따릅니다 (v1.0.0의 ET-004 "전송 중 리셋"도 넣지 않음, SOR v2.0.0 5장). 따라서 5.2의 미측정 리셋 값은 **현 시점에서 수행 계획이 없는 상태로 남습니다.**

이것이 무슨 뜻인지 분명히 해 둡니다. `X_LENGTH`(0x418)의 리셋값은 upstream 문서가 `7`, RTL 분석이 `0`으로 **서로 어긋난 채 측정되지 않았습니다.** SFR 문서에 근거 등급 **R**로 적혀 있으며, 누구도 확인하지 않은 상태로 인계됩니다. 나중에 드라이버가 이 값을 전제로 동작하거나 레지스터 사양에 의문이 생기면, **그때 측정 대상이 됩니다.**

5.4 SOR 연계 — Sanity TC → SOR v2.0.0 테스트

SOR v2.0.0 3장 "관련 Sanity" 열을 Sanity 쪽에서 본 표입니다. "같은 동작"은 Sanity와 SOR 테스트가 같은 값을 봅니다. "일부 겹침"은 Sanity가 그 테스트의 한 부분만 봅니다.

Sanity TC	같은 동작 (원래 ID)	일부 겹침 (원래 ID)
`SAN-001`	**T-01** (`REG-001`, `REG-002`, `REG-004`, `REG-005`, `REG-020`)	T-14 — 레지스터 값만, 실제 버스 신호는 아님 (`AXI-007`)
`SAN-002`	**T-01** (`REG-003`)	—

Sanity TC	같은 동작 (원래 ID)	일부 겹침 (원래 ID)
SAN-003	—	T-09 — IRQ_MASK 리셋값만 (IRQ-003). 나머지 9개는 SOR에 대응 없음
SAN-007	**T-02** (REG-021, REG-022)	T-13 — SG_ADDRESS 레지스터의 아래 3비트만 (DESC-008)
SAN-004	**T-03** (REG-007, REG-015, DESC-001, DESC-002, DESC-003, DESC-007, FUNC-001, FUNC-009; AXI-001은 관측만)	T-07 — 128 B 분할·순서 관측 (FUNC-006, FUNC-007) · T-10 — 완료 비트 set만 (REG-013)
SAN-005	**T-04** (DESC-004, FUNC-002; DESC-009는 관측만)	T-06 — 3장만 (FUNC-004) · T-08 — 마지막 장 알림, 완료 후 DESCRIPTOR_ID (DESC-005, DESC-006, REG-016)
SAN-006	—	T-09 — 마스크 0x1만 (IRQ-002, IRQ-003, IRQ-004, IRQ-005) · T-08 — 완료 이벤트, 완료 후 DESCRIPTOR_ID (IRQ-001, DESC-006, REG-016)

Sanity와 전혀 겹치지 않는 SOR 테스트: T-05, T-07(판정 부분), T-10(규칙·예약 부분), T-11, T-12, T-14(버스 신호), T-15. 이 테스트들은 Sanity 통과 여부와 상관없이 처음부터 검증해야 합니다.

> ⚠️ **Sanity 결과를 SOR 테스트의 통과 근거로 쓰지 않습니다.** v1.0.0 SOR의 원칙("Sanity는 회귀 게이트이지 요구사항 검증 근거가 아니다")은 v2.0.0에서도 같습니다. 위 표는 "같은 확인을 이미 한 번 해 보았다"는 참고 정보입니다.

5.5 읽기 금지·비교 금지 레지스터

Sanity와 SOR 모두에 적용되는 규칙입니다. **새 체크를 만들 때 반드시 지킵니다.**

주소	레지스터	규칙	이유
0x450	PARTIAL_TRANSFER_ID	**읽기 금지**	읽기만 해도 내부 FIFO가 하나 빠집니다(pop). 이 IP에서 읽기에 부작용이 있는 레지스터는 이것 하나뿐입니다 (SFR 4.5.5)
0x434 ~ 0x444	디버그군 (CURRENT_DEST_ADDRESS, CURRENT_SRC_ADDRESS, DBG_STATUS, DBG_IDS0/1)	**기대값 비교 금지**	고정 리셋값이 없습니다. 값의 타이밍이 버스 핸드셰이크에 묶여 있고, DBG_IDS0/1은 Gray 인코딩입니다 (SFR 4.7)

> ⚠️ **"전 레지스터 덤프" 방식의 테스트를 만들지 마십시오.** 주소를 차례로 다 읽으면 0x450에서 내부 상태가 망가집니다. 지금 구성에서는 FIFO가 비어 있어 피해가 없지만, 구성이 바뀌면 바로 문제가 됩니다.

Sanity에서는 두 영역을 **아예 읽지 않습니다. SOR v2.0.0에서는** 디버그군을 **"관찰 기록"으로만** 읽습니다 (값을 남기되 합격 기준에 쓰지 않음, SOR 2.1절). 0x450은 SOR에서도 읽지 않습니다 (T-02 6단계).

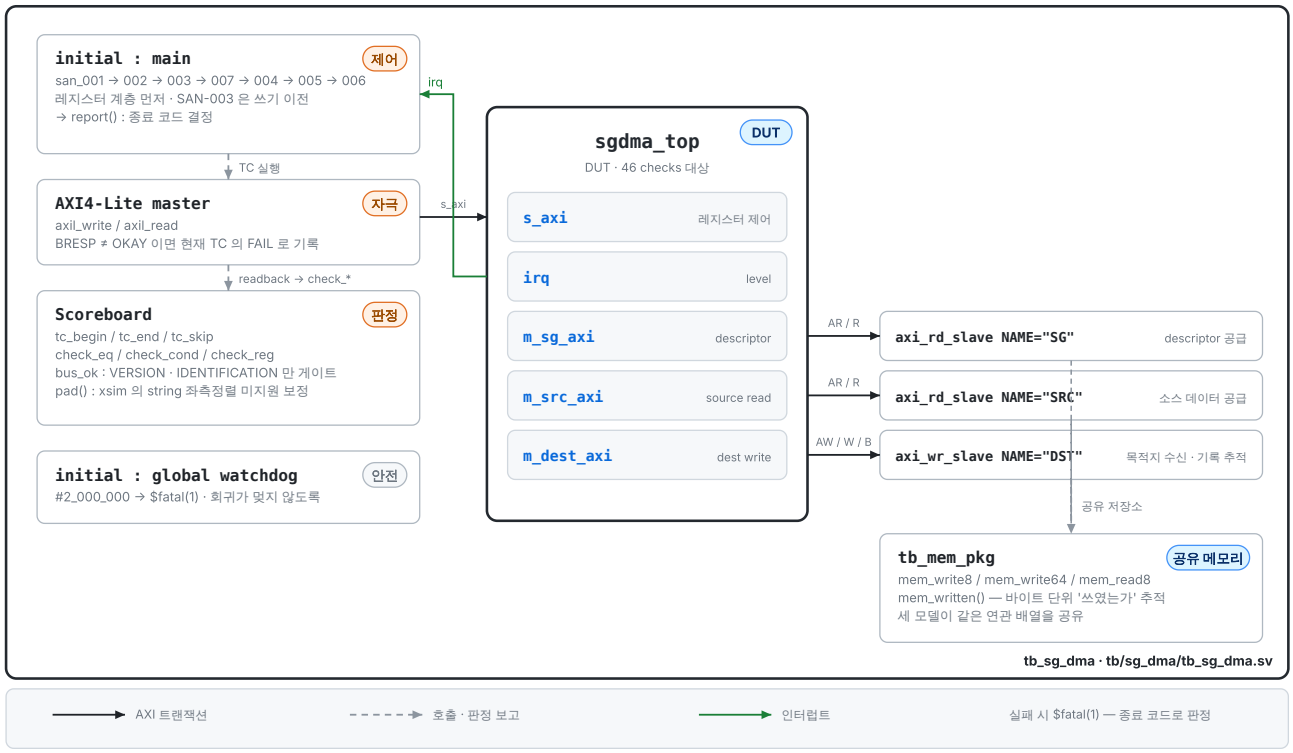
---

6. TB 구조

6.1 전체 구성

Fig.1 — Sanity Testbench 구성

tb_sg_dma · 세 메모리 모델이 tb_mem_pkg 의 같은 연관 배열을 공유하므로, 별도 BFM 없이 MM→MM 전 경로를 자급자족으로 검증한다.



mem_written() 이 바이트 단위로 '쓰였는지'를 추적하므로, 미초기화 영역을 읽어 우연히 값이 맞는 경우와 실제로 쓰인 경우를 구분할 수 있다 — tail 체크가 여기에 의존한다.

세 메모리 모델이 `tb_mem_pkg` 의 같은 연관 배열(주소를 키로 쓰는 배열)을 함께 씁니다. 그래서 소스에 쓴 패턴을 SG·DST 모델이 그대로 봅니다. descriptor를 `mem_write64()` 로 심으면 SG 포트로 읽힙니다. 별도 BFM(버스 동작 모델) 없이 MM→MM 전체 경로를 자체적으로 검증할 수 있는 이유입니다.

`mem_written()` 이 바이트마다 "쓰였는지"를 기록합니다. 그래서 초기화 안 된 영역을 읽어 우연히 값이 맞는 경우와, 실제로 쓰인 경우를 구분할 수 있습니다. `tail` 체크가 이것에 기대고 있습니다.

6.2 스코어보드

스코어보드는 체크 결과를 모아 판정하는 부분입니다.

함수	역할
`tc_begin(name, desc)`	TC 시작. 카운터 초기화
`tc_end()`	TC 종료. 결과를 큐에 push
`tc_skip(name, desc, why)`	TC 건너뜀 기록
`check_eq(name, got, exp)`	32비트 값 비교
`check_cond(name, cond, detail)`	참/거짓 조건
`check_reg(name, addr, exp)`	읽기 + 비교를 한 번에
`report()`	요약 출력 및 종료 코드 결정

`pad()` 함수가 있는 이유: xsim은 `string` 타입에 `%-Ns` 좌측 정렬을 적용하지 않습니다. 그대로 쓰면 출력이 오른쪽 정렬되어 읽기 어렵습니다.

```
function automatic string pad(input string s, input int n);
  string r = s;
  while (r.len() < n) r = {r, " "};
  return r;
endfunction
```

6.3 AXI4-Lite 마스터

`axil_write`는 AW/W 채널을 **동시에 구동**하고, 양쪽 핸드셰이크가 끝날 때까지 기다립니다. `BRESP`가 `OKAY`가 아니면 현재 TC의 FAIL로 기록합니다.

```
if (s_axi_bresp !== 2'b00) begin
  cur_fail++;
  $display("    [FAIL] AXI-Lite write resp : addr=0x%03h bresp=%0d", addr, s_axi_bresp);
end
```

이 IP는 에러 응답을 내지 않으므로, 이 검사는 평소에는 조용합니다. 레지스터 접근이 어떤 이유로 에러를 내기 시작하면 바로 드러나는 안전망입니다.

---

7. 유지보수

7.1 파라미터를 바꿨을 때

바꾼 파라미터	고쳐야 할 기대값
`DMA_AXI_ADDR_WIDTH` 32 → 64	`SAN-007` 체크 6~8 (HIGH 레지스터가 **RW가 됨**), TB의 `AW` localparam
`AXI_AXCACHE` / `AXI_AXPROT`	`EXP_INTERFACE_DESCRIPTION_2`
`DMA_DATA_WIDTH_*`	`EXP_INTERFACE_DESCRIPTION_1`, `SAN-007` 정렬 체크, burst beat 수
`MAX_BYTES_PER_BURST`	`EXP_INTERFACE_DESCRIPTION_1`, 분할 동작
`DMA_2D_TRANSFER` 0 → 1	`SAN-007` 체크 9 (`Y_LENGTH`가 활성), `EXP_INTERFACE_DESCRIPTION_1`
`FRAMELOCK` 0 → 1	`SAN-007` 체크 1·10
`CYCLIC` 0 → 1	`SAN-007` 체크 2
`ID`	`EXP_PERIPHERAL_ID`
`FIFO_SIZE`	`ID_WIDTH` 변화 → 디버그 레지스터 (현재 체크 없음)

`INTERFACE_DESCRIPTION_1`은 거의 모든 파라미터 변경에 영향을 받습니다. 반대로, 파라미터를 바꿨는데 이 값이 그대로면 **빌드에 반영되지 않은 것**입니다.

파라미터를 바꾸면 SOR v2.0.0의 해당 기대값(T-01, T-02, T-03 등)도 같이 고쳐야 합니다.

7.2 체크를 추가할 때

규칙	이유
리셋값 체크는 **`SAN-003`에만**	쓰기가 일어나기 전에 실행되도록 순서가 고정되어 있음

규칙	이유
`0x450` 을 읽는 체크는 금지	읽기가 FIFO를 pop (5.5)
디버그군(`0x434` ~ `0x444`) 기대값 비교 금지	고정값 없음. Gray 인코딩 (5.5)
ADI가 약속하지 않은 동작(구현 특성)을 체크하면 **문서에 "구현 특성 체크"로 표시**	실패했을 때 RTL 버그로 오해하지 않게 (3.8)
시퀀스가 필요하면 **Sanity가 아님**	빠름 / 자동 판정 조건에 안 맞음 → SOR 테스트
헤더 주석의 체크 개수를 **같이 수정**	안 고치면 문서와 어긋남 (실제로 어긋난 적 있음)

TB 헤더에 TC별 체크 수와 합계를 적어 두었습니다. 로그의 `(46/46 checks)` 와 맞춰 보면 빠진 것을 바로 알 수 있습니다.

```
//   SAN-005 : 3-descriptor gather        (6 checks)
//   SAN-006 : IRQ & status               (7 checks)
//   ------------------------------------------
//   Total                               (46 checks)
```

7.3 문서 동기화

TB와 아래 문서는 같이 움직여야 합니다.

변경	같이 고칠 것
기대값	`SGDMA-SFR-rel-v2.0.0` §3 · §4 · 부록 A, 해당 SOR v2.0.0 테스트의 통과 기준
체크 추가/삭제	이 문서 2.1 · 3장 · 부록 B, TB 헤더
커버 범위 변화	이 문서 5장, `SGDMA-SOR-rel-v2.0.0` 3장 "관련 Sanity" 열과 6.3절
구현 특성 체크의 처리 방식 (예: 경고로 낮춤)	이 문서 2.3 · 3.8 · 4.3 · 부록 B, TB 헤더
레지스터 명칭	SFR 문서, TB localparam, 체크 문자열

부록 A — 실측 로그 전문

2026-10-06 실행. `sgdma_top` 리네임 · `AXI_AXCACHE` / `AXI_AXPROT` 명시 · 레지스터 명칭 통일이 모두 반영된 상태입니다.

```
=======================================================
  SG DMA sanity test   (SGDMA-SANITY-v1.0)
=======================================================

[SAN-001] Register identity
    [ ok ] VERSION                          0x00040565
    [ ok ] IDENTIFICATION                   0x444d4143
    [ ok ] PERIPHERAL_ID                    0x00000000
    [ ok ] INTERFACE_DESCRIPTION_1          0x38070303
    [ ok ] INTERFACE_DESCRIPTION_2          0x00000000

[SAN-002] Scratch R/W
    [ ok ] SCRATCH = 0xA5A55A5A             0xa5a55a5a
    [ ok ] SCRATCH = 0x5A5AA5A5             0x5a5aa5a5
    [ ok ] SCRATCH = 0x00000000             0x00000000

[SAN-003] Reset defaults
    [ ok ] CONTROL                          0x00000000
    [ ok ] IRQ_MASK                         0x00000003
    [ ok ] IRQ_PENDING                      0x00000000
    [ ok ] IRQ_SOURCE                       0x00000000
```

```
    [ ok ] FLAGS                              0x00000002
    [ ok ] TRANSFER_DONE                      0x00000000
    [ ok ] TRANSFER_ID                        0x00000000
    [ ok ] DEST_ADDRESS                       0x00000000
    [ ok ] SRC_ADDRESS                        0x00000000
    [ ok ] SG_ADDRESS                         0x00000000

[SAN-007] Parameter-masked bits
    [ ok ] CONTROL[3] masked (FRAMELOCK=0)    0x00000007
    [ ok ] FLAGS[0] masked (DMA_CYCLIC=0)     0x00000006
    [ ok ] DEST_ADDRESS 8B aligned            0xfffffff8
    [ ok ] SRC_ADDRESS 8B aligned             0xfffffff8
    [ ok ] SG_ADDRESS 8B aligned              0xfffffff8
    [ ok ] DEST_ADDRESS_HIGH tied 0           0x00000000
    [ ok ] SRC_ADDRESS_HIGH tied 0            0x00000000
    [ ok ] SG_ADDRESS_HIGH tied 0             0x00000000
    [ ok ] Y_LENGTH tied 0 (2D=0)             0x00000000
    [ ok ] FRAMELOCK_CONFIG tied 0            0x00000000
    [ ok ] unmapped reads return 0
    restored CONTROL / FLAGS / address registers

[SAN-004] Single descriptor copy (256 B)
    desc @0x00001000 : id=0xa0 flags=11 src=0x00010000 dst=0x00020000 len=256 next=0x00000000
[3195000]   SG AR  addr=0x00001000 beats=6 size=8B
[3315000]  SRC AR  addr=0x00010000 beats=16 size=8B
[3495000]  DST AW  addr=0x00020000 beats=16 size=8B
[3495000]  SRC AR  addr=0x00010080 beats=16 size=8B
[3705000]  DST AW  addr=0x00020080 beats=16 size=8B
    [ ok ] SAN-004 : irq asserted
    [ ok ] data : 256 bytes 0x00010000 -> 0x00020000
    [ ok ] tail : no overrun past 0x00020100
    [ ok ] TRANSFER_DONE bit set

[SAN-005] 3-descriptor chain, gather
    desc @0x00001100 : id=0xb0 flags=00 src=0x00011000 dst=0x00021000 len=128 next=0x00001140
    desc @0x00001140 : id=0xb1 flags=00 src=0x00013000 dst=0x00021080 len=256 next=0x00001180
    desc @0x00001180 : id=0xb2 flags=11 src=0x00015000 dst=0x00021180 len=64 next=0x00000000
[4215000]   SG AR  addr=0x00001100 beats=6 size=8B
[4295000]   SG AR  addr=0x00001140 beats=6 size=8B
[4335000]  SRC AR  addr=0x00011000 beats=16 size=8B
[4375000]   SG AR  addr=0x00001180 beats=6 size=8B
[4515000]  DST AW  addr=0x00021000 beats=16 size=8B
[4515000]  SRC AR  addr=0x00013000 beats=16 size=8B
[4695000]  SRC AR  addr=0x00013080 beats=16 size=8B
[4725000]  DST AW  addr=0x00021080 beats=16 size=8B
[4875000]  SRC AR  addr=0x00015000 beats=8 size=8B
[4915000]  DST AW  addr=0x00021100 beats=16 size=8B
[5105000]  DST AW  addr=0x00021180 beats=8 size=8B
    [ ok ] SAN-005 : irq asserted
    [ ok ] seg0 : 128 bytes 0x00011000 -> 0x00021000
    [ ok ] seg1 : 256 bytes 0x00013000 -> 0x00021080
    [ ok ] seg2 : 64 bytes 0x00015000 -> 0x00021180
    [ ok ] tail : no overrun past 0x000211c0
    [ ok ] DESCRIPTOR_ID = last of chain      0x000000b2

[SAN-006] IRQ & status
    desc @0x00001200 : id=0xc5 flags=11 src=0x00018000 dst=0x00028000 len=64 next=0x00000000
    [ ok ] IRQ_MASK = 0x1 (EOT unmasked)      0x00000001
[5595000]   SG AR  addr=0x00001200 beats=6 size=8B
[5715000]  SRC AR  addr=0x00018000 beats=8 size=8B
[5815000]  DST AW  addr=0x00028000 beats=8 size=8B
    [ ok ] SAN-006 : irq asserted
```

```
    [ ok ] IRQ_PENDING[1] set
    [ ok ] IRQ_PENDING cleared                  0x00000000
    [ ok ] IRQ_SOURCE[0] survives W1C           0x00000001
    [ ok ] irq de-asserted after W1C
    [ ok ] DESCRIPTOR_ID                        0x000000c5

=========================================================
  SANITY SUMMARY
=========================================================
  SAN-001  Register identity               PASS  (5/5)
  SAN-002  Scratch R/W                     PASS  (3/3)
  SAN-003  Reset defaults                  PASS  (10/10)
  SAN-007  Parameter-masked bits           PASS  (11/11)
  SAN-004  Single descriptor copy (256 B)  PASS  (4/4)
  SAN-005  3-descriptor chain, gather      PASS  (6/6)
  SAN-006  IRQ & status                    PASS  (7/7)
---------------------------------------------------------
  SANITY: 7 PASS / 0 FAIL / 0 SKIP     (46/46 checks)
=========================================================
*** SANITY PASSED ***
$finish called at time : 6305 ns
```

부록 B — 체크 ↔ SFR 근거 대조표

46개 체크 전부가 SFR 문서의 어느 절에서 기대값을 가져왔는지입니다. 기대값을 고칠 때는 양쪽을 같이 고쳐야 합니다. SFR 절 번호는 SFR rel-v1.0.0 기준입니다. "성격" 열의 **구현 특성**은 3.8을 따릅니다 (TB 판정은 다른 체크와 같음).

TC	#	체크	기대값	SFR 절	성격
SAN-001	1	`VERSION`	`0x0004_0565`	4.1.1	게이트 (`bus_ok`)
	2	`IDENTIFICATION`	`0x444D_4143`	4.1.4	게이트 (`bus_ok`)
	3	`PERIPHERAL_ID`	`0x0000_0000`	4.1.2	빌드 구성
	4	`INTERFACE_DESCRIPTION_1`	`0x3807_0303`	4.1.5	빌드 구성
	5	`INTERFACE_DESCRIPTION_2`	`0x0000_0000`	4.1.6	빌드 구성
SAN-002	6~8	`SCRATCH` 3회	기입값	4.1.3	
SAN-003	9	`CONTROL`	`0x0000_0000`	4.3.1	
	10	`IRQ_MASK`	**`0x0000_0003`**	4.2.1	
	11	`IRQ_PENDING`	`0x0000_0000`	4.2.2	
	12	`IRQ_SOURCE`	`0x0000_0000`	4.2.3	
	13	`FLAGS`	**`0x0000_0002`**	4.4.3	
	14	`TRANSFER_DONE`	`0x0000_0000`	4.5.1	
	15	`TRANSFER_ID`	`0x0000_0000`	4.4.1	
	16	`DEST_ADDRESS`	`0x0000_0000`	4.4.4	
	17	`SRC_ADDRESS`	`0x0000_0000`	4.4.5	
	18	`SG_ADDRESS`	`0x0000_0000`	4.6.1	
SAN-007	19	`CONTROL[3]` 마스크	`0x0000_0007`	4.3.1 / 4.8	
	20	`FLAGS[0]` 마스크	`0x0000_0006`	4.4.3 / 4.8	

TC	#	체크	기대값	SFR 절	성격
	21~23	주소 3종 8B 정렬	`0xFFFF_FFF8`	4.4.4 / 4.6.1	
	24~26	`*_ADDRESS_HIGH` 3종	`0x0000_0000`	4.8 / §10	
	27	`Y_LENGTH`	`0x0000_0000`	4.8	
	28	`FRAMELOCK_CONFIG`	`0x0000_0000`	4.8	
	29	미매핑 2주소	`0x0000_0000`	2.1	
SAN-004	30	irq asserted	—	§6	
	31	data 256 B	바이트 일치	§5	
	32	tail 미기입	—	—	
	33	`TRANSFER_DONE` 비트	`[3:0] != 0`	4.5.1	
SAN-005	34	irq asserted	—	6.2	
	35~37	seg0 / seg1 / seg2	바이트 일치	§5	
	38	tail 미기입	—	—	
	39	`DESCRIPTOR_ID`	`0x0000_00B2`	**4.6.2**	**구현 특성**
SAN-006	40	`IRQ_MASK`	`0x0000_0001`	4.2.1	
	41	irq asserted	—	6.1	
	42	`IRQ_PENDING[1]`	`[1] = 1`	4.2.2	
	43	`IRQ_PENDING` clear	`0x0000_0000`	4.2.2	
	44	**`IRQ_SOURCE[0]` 잔존**	**`0x0000_0001`**	**4.2.3**	
	45	irq 하강	`irq = 0`	6.1	
	46	`DESCRIPTOR_ID`	`0x0000_00C5`	4.6.2	**구현 특성**

근거 등급이 M(실측)인 값은 이 46개로 확정된 것들입니다. SFR 문서에서 R(RTL 분석)로 남은 값은 이 표에 없습니다. 단, 39·46번은 실측(M)이지만 **ADI가 약속한 동작이 아니므로** 다음 RTL에서도 같다는 보장이 없습니다.

부록 C — 설계 결정 반영 이력

C.1 코드에 반영된 결정 (2026-10-06)

#	결정	TB 영향	검증
1	`AXI_AXCACHE` = `4'b0000` 명시	기대값 불변	명시 전후 **46/46 + 타임스탬프 19개 전부 동일**
2	`DMA_AXI_ADDR_WIDTH` = 32 확정	`AW` localparam, `SAN-007` 체크 6~8 유지	—
3	모듈명 `sgdma_top`	DUT 인스턴스화 1줄	46/46
4	래퍼 파라미터 38개 명시	기대값 불변	46/46
5	AXI ID / LOCK 포트 노출	신호 선언 추가, 입력은 tie-off	**46/46 + 타임스탬프 동일**
6	레지스터 명칭 upstream 통일	localparam · 체크 문자열	46/46
7	Sanity 보강 2건	**재조정 — 아래 C.2**	—

결정 1과 5는 "동작을 바꾸지 않는 변경"이었음을 사이클 단위로 확인했습니다. 통과 개수만 보면 "다른 타이밍으로 우연히 통과"를 배제할 수 없습니다. 그래서 AXI 트랜잭션 발생 시각 19개를 전부 대조했습니다.

C.2 결정 7 — 재조정

원안: `X_LENGTH`(0x418)와 `ACTIVE_TRANSFER_ID`(0x42C)의 리셋값 체크를 `SAN-003`에 추가 (46 → 48).

재조정 (2026-10-06 후속 논의): **Sanity는 현행 46 checks 범위를 유지**한다. 이어서 SOR에서도 `RST` **요구사항 6건을 삭제**하고 3-1 Reset / Clock을 "Sanity Test에서 커버됨 · 해당 사항 없음"으로 정리하기로 했다. 따라서 **미측정 리셋값은 현 시점에서 수행 계획이 없다.** (SOR v2.0.0도 이 결정을 유지한다.)

판단 근거: Sanity의 역할은 회귀 게이트이지 커버리지 도구가 아니다. 리셋값 검증은 현 단계에서 **투자 대비 우선순위가 낮다**고 판단해 Sanity와 SOR 양쪽에서 범위 밖으로 둔다.

해소되지 않은 상태: `X_LENGTH` 리셋값은 **upstream 문서(`7`)와 RTL 분석(`0`)이 어긋난 채 미측정**입니다. SFR 문서에서 근거 등급 **R**로 표기되어 있으며, **수행 계획 없이 그대로 인계됩니다.** 측정 전까지 확정된 값이 아니라는 점을 문서가 계속 드러내는 것이 이 등급 체계의 역할입니다.

C.3 TB 자체의 수정 이력

시점	내용
2026-10-01	초기 45 checks. `INTERFACE_DESCRIPTION_2` 기대값 `0x30`으로 FAIL → **실측 `0x00`으로 정정**
2026-10-01	SKIP 로직 수정 — 구성 레지스터 불일치가 전체 TC를 SKIP시키던 문제. `bus_ok` 플래그 도입
2026-10-01	`pad()` 함수 추가 — xsim이 `string`에 `%-Ns` 좌측 정렬을 적용하지 않음
2026-10-02	`SAN-005`에 `DESCRIPTOR_ID` 체크 추가 (45 → **46**)
2026-10-02	AXI ID / LOCK 포트 신호 추가
2026-10-06	모듈명 · 레지스터 명칭 · 주석 정리

> **2026-10-01의 두 사건이 이 TB의 설계 판단 둘을 만들었습니다.** 기대값 오류가 전체 SKIP을 일으킨 경험에서 `bus_ok` 게이트가 나왔습니다. 정적 분석만으로 두 문서가 같은 틀린 값에 도달한 경험에서 SFR 문서의 **근거 등급(M/R/D)** 체계가 나왔습니다.

v2.0.0(2026-10-16)에서는 TB를 고치지 않았습니다. 문서 개정 내용은 1.6과 아래 개정 이력에 있습니다.

개정 이력

버전	일자	내용
v0.2	2026-10-01	항목 정의서 — TC 7건 / 45 checks
rel-v1.0.0	2026-10-06	**검증팀 인계본** — 46 checks 확정, 실측 로그 수록, **커버 범위와 한계(§5) 신설**, 결정 7 재조정 기록, TB 구조·유지보수 지침 추가
rel-v2.0.0	**2026-10-16**	**SOR rel-v2.0.0(테스트 15개)에 맞춘 문서 개정. TB·체크 46개·기대값·실측 로그는 그대로.** ① SOR 연결을 원래 61건 ID 기준에서 T-01 ~ T-15 기준으로 다시 매김(원래 ID는 괄호로 유지, 5.4 신설) ② `DESCRIPTOR_ID` 완료 후 값 체크(`0xB2`, `0xC5`)를 "구현 특성 체크"로 표시하고 해석 규칙·경고 강등 권장 추가(3.8 신설) ③ "Sanity가 커버하지 않는 것"을 SOR v2.0.0 기준으로 갱신(5.2·5.3) ④ 읽기·비교 금지 레지스터 규칙을 5.5로 모음 ⑤ 문서 참조를 v2.0.0으로 변경 ⑥ 처음 보는 사람용으로 다시 씀(1장 비유·용어 풀이, 체크별 "틀리면" 열). 근거: `SGDMA-SOR-INTEGRATION-v1.0` 7장 #1~#3